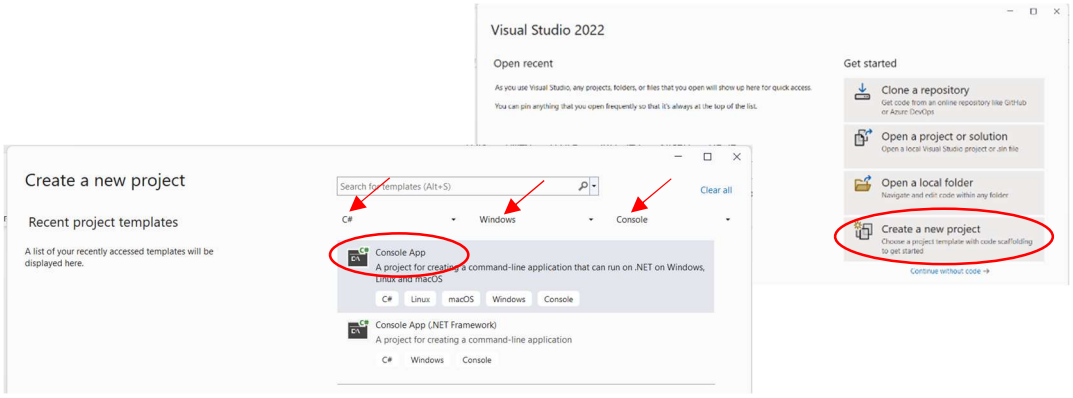
בס"ד

פרויקט C# - שלב 1 – יצירת פרויקטים לשכבת DAL

1. יצירת פרויקט

צרו פרויקט חדש **Create a new Project** (או בתפריט **File=>New=>Project...**)
ובחרו בניית פרויקט מסוג מסוג **Console Application**:
נ.ב. בשום פנים ואופן לא לבחור סוג **Console Application (.NET Framework)**



קנפגו את הפרויקט לפי הסדר כפי שמופיע כאן – קודם כל שם הפרויקט ורק לאחר מכן שם הפתרון!

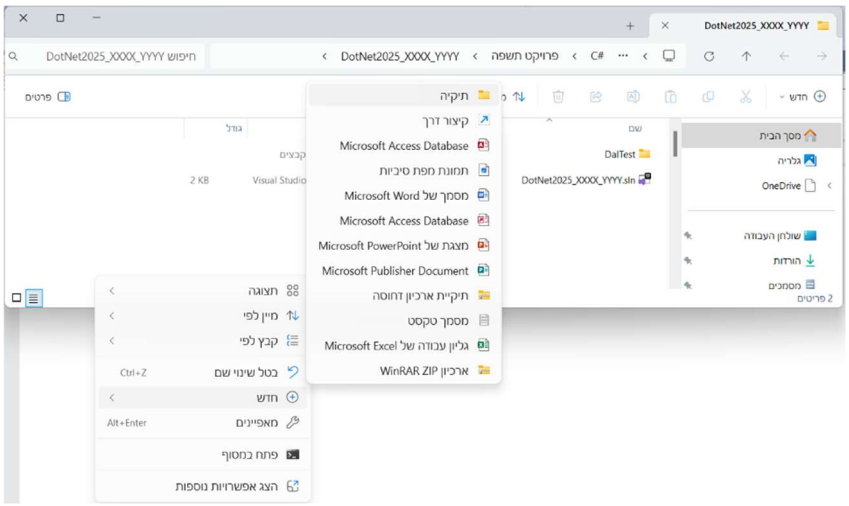
1. שם הפרויקט (**Project**) יהיה **DalTest** (פרויקט קונסול לבדיקת שכבת הנתונים)
2. שם הפתרון (**Solution**) יהיה **DotNet2027_XXXX_YYYY**
   כאשר XXXX , YYYY הן 4 הספרות האחרונות של ת.ז. של כל אחת מבנות הזוג.
3. המיקום (**Location**) – איפה תמוקם תיקיית הפרויקט, יש לבחור מיקום נוח עבורכן
4. חשוב!! אין לסמן בתיבת סימון **Place solution and project in the same directory**!

לחצו על כפתור **Next**. בחלון הבא וודאו ש-**Target Framework** מוגדר כ- **NET 6.0**
לפחות ולחצו על כפתור **Create**.

2. יצירת תיקיית הרצה

הסבר: כברירת מחדל, עבור כל פרויקט שמוסיפים ל solution נוצרת תיקיית bin עבור קבצי הריצה של אותו פרויקט.
ה solution שלנו יהיה מורכב מפרויקטים רבים ונרצה לרכז את כל קבצי הריצה במקום אחד, בתיקייה הראשית של ה solution וכך למנוע חלק מהבעיות וקושי של מעקב אחר קבצי הריצה בהמשך הדרך.

לצורך כך, בשלב זה ניצור את תיקיית ההרצה המשותפת ובהמשך נגדיר אותה כתיקיית ההרצה של כל אחד מהפרויקטים.
בתיקיה הראשית של ה-Solution יש ליצור תת-תיקיה בשם bin. היצירה חייבת להתבצע בסייר קבצים - File Explorer, בשום פנים ואופן אסור "ליצור" את התיקיה מתוך VS
כדי להגיע לתיקיית ה solution מתוך VS, יש ללחוץ במקש ימני על ה solution ולבחור "Open Folder in File Explorer"



3. הוספת שני פרויקטים נוספים ל solution:

עכשיו יש להוסיף ל-Solution שני פרויקטים נוספים (במהלך הוספה יש לוודא שהפרויקטים נפתחים עם Framework בגרסה NET 6.0. לפחות).

יש להוסיף את הפרויקטים בהתאם ל"הנחיות לפתיחת פרויקט":

1. פרויקט **DalFacade** מסוג Class Library,
   הערך תחת RootNamespace (Default namespace) צריך להיות **ריק**
2. פרויקט **DalList** מסוג Class Library,
   הערך תחת RootNamespace (Default namespace) צריך להיות "**Dal**"
3. פרויקט **DalTest** (שיצרנו ביצירת ה solution) מסוג Console App, אין צורך לשנות את הערך של RootNamespace (Default namespace)

4. קישור בין הפרויקטים

א. נרצה שפרויקט DalList יוכל לגשת בקוד לישויות ולמחלקות הציבוריות שיוגדרו בפרויקט DalFacade ולכן ניצור הפניה (reference) ממנו אליו:
עמדו עם עכבר ימני על הפרויקט DalList ואז Add → Project Reference
בחלון שיפתח סמנו את הפרויקט DalFacade, ואז לחצו OK
ודאו בקובץ המאפיינים של הפרויקט שהטקסט שלו התעדכן לאחר הוספת הרפרנס:

```
<ItemGroup>
  <ProjectReference Include="..\DalFacade\DalFacade.csproj"/>
</ItemGroup>
```

חשוב שתבינו את המשמעות של רפרנס בין פרויקט לפרויקט:
פרויקט DalList מכיר את ההגדרות שנמצאות ב-DalFacade ולא ההיפך!
וכן, שכאשר נבנה/נקמפל את פרויקט DalList אז באופן אוטומטי הוא קודם כל יבנה/יקמפל את הפרויקט שהוא תלוי בו (DalFacade) .

ב. נרצה שפרויקט DalTest יוכל לגשת בקוד לישויות ולמחלקות הציבוריות שיוגדרו בפרויקט DalFacade וכן יוכל לגשת לממשקים הציבוריים שמוגדרים ב-DalList, לכן ניצור הפניה (reference) ממנו אליהם:
עמדו עם עכבר ימני על הפרויקט DalTest ואז Add → Project Reference
בחלון שיפתח סמנו את 2 הפרויקטים DalFacade, DalList, ואז לחצו OK
ודאו בקובץ המאפיינים של הפרויקט שהטקסט שלו התעדכן לאחר הוספת הרפרנס:

```
<ItemGroup>
  <ProjectReference Include="..\DalFacade\DalFacade.csproj"/>
  <ProjectReference Include="..\DalList\DalList.csproj"/>
</ItemGroup>
```

הסבר כללי - 3 הפרויקטים של שכבת ה-DAL

1. פרויקט בשם **DalFacade**- חזית (facade) הנתונים. הפרויקט יכיל:
   a. הגדרות של הישויות הקיימות, הגדרה נפרדת לכל ישות בקובץ נפרד.
   b. הגדרות של המתודות שניתן לבצע על הנתונים. נגדיר את אוסף המתודות בצורה מסודרת, כממשק .(Interface)
2. פרויקט בשם **DalList**- בסיס הנתונים, שכבת הנתונים. הפרויקט יכיל עבור כל ישות שמוגדרת ב-:DalFacade
   a. אוסף הנתונים שיש מכל ישות (כל אוסף שמור בתור רשימות בזיכרון הפנימי מסוג **List**). בשלב זה הנתונים יהיו שמורים ברשימות (Lists) בתוך הפרויקט DalList. בשלבים הבאים הנתונים יהיו שמורים בזיכרון חיצוני (קבצי XML), בפרויקט נפרד בשם DalXml.
   b. מחלקה שתממש את המתודות שניתן לבצע על כל ישות ע"י פנייה לאוסף נתונים מסוג **List**, של כל ישות. מימוש הממשק שמוגדר עבור אותה ישות ב-DalFacade.
3. פרויקט בשם **DalTest**- פרויקט שיכיל תוכנית ראשית שמטרתה:
   a. לאתחל את בסיס הנתונים השמור בפרויקט DalList (תמלא את כל רשימות הישויות בנתונים)
   b. לבדוק את תקינות כל המתודות שניתן לבצע על הנתונים השמורים ברשימות, דרך מהממשקים המוגדרים בפרויקט .DalFacade
   c. כשנריץ את שלב 1, בעצם נריץ את הפרויקט DalTest והוא יתקשר עם שני הפרויקטים של שכבת הנתונים (DalFacade, DalList).

5. יצירת ישויות הנתונים

<u>הנחיות כלליות</u>:

- חובה לפרמט את הקוד בהתאם - הזחות, שורות רווח, ריווח בתוך השורות
- כל השמות (של המחלקות, השדות, התכונות, המתודות) חייבות להיות באנגלית ובעלי משמעות בהתאם לתפקיד המחלקה, המבנה, התכונה, המתודה
- שמות כל הסוגים, כמו כן השדות, התכונות, והמתודות עם הרשאה public יש להגדיר בפורמט PascalCase
- שמות כל השדות והתכונות בהרשאה private או internal יש להגדיר בפורמט _camelCase
- שמות כל המתודות בהרשאה private או internal, הפרמטרים, והמשתנים המקומיים יש להגדיר בפורמט camelCase
- חובה לתעד את כל הסוגים, המתודות, והתכונות בעזרת תיעוד מפורמט (///)
- חובה לתעד את הקוד אם איננו ברור מאליו ממבנה הקוד (למשל במקרה של חישובים מתמטיים שאינם בסיסיים)

<u>ישויות הנתונים</u>:
צרו בפרויקט DalFacade תת-תיקיה בשם DO:
לחיצה ימנית על הפרויקט -> Add -> New Folder

בס"ד

מלבד קבצי הגדרת הישויות, בספריה זו גם נשמור גם את קובץ האנומרציות הנחוצות בהגדרת ישויות הנתונים. בשלב זה נוסיף קובץ אנומרציות ריק בשם :**Enums.cs**
להוספת קובץ אנומרציות ריק לספריית **DO**:

1. עמדו עם עכבר ימני על הספריה **DO**
2. לחצו על **Add → Class**
3. כתבו את שם הקובץ **Enums.cs** ולחצו על **Add**
4. יווצר לכם קובץ cs בשם **Enums.cs**
5. הסירו מראש הקובץ את רשימת הusing- הקיימת, משום שאין צורך לציין using גלובליים. והם יצוינו אוטומטית בקובץ נפרד.
6. שימו לב שמרחב השמות של הקובץ נקרא באופן אוטומטי **DO** כמו שם הספריה שייצרתם.
7. הוסיפו סימן נקודה-פסיק "**;**" בסוף השורה: "namespace DO;"
   a. שימו לב שהצומדיים של הבלוק {...} של מרחב השמות נמחקים אוטומטית ע"י ויז'ואל סטודיו, וכל התוכן של הבלוק מוזח שמאלה בהתאם.
   b. וודאו שהשורה הזו נמצאת בראש הקובץ (ראשונה בקובץ)
8. רוקנו את הקובץ מהגדרת המחלקה שנוצרה. בהמשך, כאשר נגדיר את הישויות עצמן, נמלא את הקובץ בהגדרות של enums על פי הצורך.

כעת ניגש ליצירת טיפוס חדש עבור כל ישות נתונים (ישות DO) נדרשת.

<u>הנחיות כלליות להגדרת ישות ותכונותיה:</u>
נרצה לחסוך בהעתקות במעבר של ישות משכבה לשכבה וכן להגן על הישויות כך שישמרו בבסיס הנתונים בצורה אותנטית. כלומר, לוודא שכאשר ישות עוברת כהפניה משכבה לשכבה, היא תעבור בצורה שלא ניתנת לשינוי (**immutable**). לכן:

1. נגדיר כל ישות כטיפוס מסוג **record** (לא class ולא struct) בהרשאת גישה **public** במידה ובכל זאת נצטרך להעתיק ישות משכבה לשכבה, נרצה שהעותק יהיה זהה בערכיו וללא הפניות.
   לכן נגדיר כל ישות כ **PDS**, ראשי תיבות של **Passive Data Structure**, כלומר ישות בעלת מבנה שטוח ופסיבי. כדי שישות תהיה **PDS** אזי כל התכונות שלה צריכות להיות מסוג **value type**. (הרחבה והסבר נוסף בסוף ההנחיה)
2. לגבי כל אחת מהתכונות עליכם להחליט אם היא צריכה להיות **value type** רגיל או **nullable value type**
3. כאמור, לישות מסוג **record**, מוגדרת באופן אוטומטי העמסה למתודה .**<u>ToString</u>** לכן אין צורך להעמיס אותה מחדש. אלא אם כן, אתם רוצים לשנות אותה (לקצר אותה וכדומה).
4. אין להוסיף מתודות נוספות להגדרת הישות - כי זו ישות PDS!
5. המתודות היחידות שעליכם להוסיף הן 2 בנאים:
   א. בנאי עם פרמטרים - שמקבל כפרמטרים, את כל ערכי התכונות (בהתחשב גם בערכי ברירת מחדל) ← אם הגדרתם את ה record בצורה שלמדנו, כבר קיים בנאי כזה אוטומטי ואין צורך להוסיף שוב!
   ב. בנאי ריק (שלא מקבל פרמטרים) - נועד לצורך עתידי שנגיע אליו בהמשך.
6. חשוב מאוד! <u>הוספת תיעוד</u>**:** הוסיפו הערות לפני הגדרת כל ישות, בהערות יש לתאר בקצרה את משמעות הישות וכן הסבר על כל אחד ממאפייניה.

נגדיר שלוש ישויות נתונים – פירוט ניתן לראות בתיאור הכללי של הפרויקט.

בס"ד

<u>ישות PDS – ישות שטוחה ופסיבית</u>

כל אחת מהישויות שנגדיר צריכה להיות **PDS**, ראשי תיבות של **Passive Data Structure**, כלומר ישות בעלת מבנה שטוח ופסיבי. על מנת שההעתקה של ישות PDS תיצור עותק זהה בערכיו וללא הפניות.

<u>מהי ישות **PDS**?</u>

- תכיל רק תכונות ולא תכיל מתודות (למעט העמסת של מתודה ToString לצורכי debug)
- תכונות אלו יהיו מטיפוסים שהם ValueType בלבד ולא ReferenceType (למעט מחרוזות - string), כלומר:
  - טיפוסים פשוטים כמו int, char, bool וכדומה
  - DateTime, TimeSpan
  - טיפוסים מסוג של אנומרציות מוגדרות - סוגי enum שיוגדרו ע"י המתכנת
  - מחרוזות (string) - היוצא מן הכלל שהינו Reference Type
    - מחרוזת מסוג string הינו סוג הפניה ש<u>לא ניתן לשינוי</u>(immutable)
- לא תכיל תכונות מטיפוס רשימה או אוסף מכל סוג שהוא
- לא תירש מאף מחלקה אחרת

אובייקטים של הישויות (הנתונים) יעברו בין השכבות של הפרויקט:

- בפרמטרים של מתודות הממשק
- בערך חוזר בודד של מתודת ממשק
- באוסף אובייקטים של ישות שמוחזר ממתודת ממשק

למשל, שכבת DAL מחזירה לשכבת שמעליה (BL - שכבה לוגית) את אוסף האובייקטים של ישות מסוימת, או ההיפך.

חשוב - מכיוון שהעתקות של מבנים (**struct**) ושל רשומות (**record**) הינן העתקות רדודות (**shallow copy**) - בזכות העובדה שהישות היא **PDS**, אזי כאשר יוצרים ממנה העתק, הוא זהה בערכיו ממש ולא מכיל כתובות/הפניות (למעט הפניות למחרוזות, שלא ניתנות לשינוי).

השילוב של ישות שמוגדרת כ-**record** ושהיא גם **PDS** יוצר את <u>היתרונות</u> הבאים לצורכי הפרויקט שלנו:

- ה-record הוא טיפוס הפניה, לכן כשמעבירים אובייקט בין שכבות, בעצם מעבירים הפניה ולא אובייקט שלם וזה יותר יעיל מבחינת ביצועי מעבד.
- מצד שני, בזכות העובדה שניצור record עם תכונות שאינן ניתנות לשינוי (immutable), אזי האובייקט שעובר בין השכבות יהיה מוגן וישמור על ערכיו
- במידה ובכל זאת רוצים לשנות שדות של אובייקט (למשל במתודת עדכון), אז בשכבה הרלוונטית ניצור העתק שלו. ומכיוון שהישות היא PDS ולא מכילה הפניות או רשימות אזי ההעתק שטוח וזהה בערכיו לאובייקט המקורי.